Safe Edges

2026

Web & APP Security Audit REPORT

ASSESSED ON , 2026

Security Assessment

SAFEEDGES.IN

Table of Contents

Project Summary

SNPIT is a Flutter-based mobile application centered on photo capture, collection, and game-like engagement. The app allows users to take and upload photos, view photo details, participate in voting battles, complete missions, manage inventory items, use camera-related features, interact with rentals and marketplace screens, and access wallet or NFT-related functionality. It also includes onboarding, profile management, email and SMS verification, notifications, advertisements, rewards, and multilingual content delivery.

The codebase is structured as a cross-platform Flutter client using `Riverpod` for state management, typed `GoRouter` routes for navigation, and generated `OpenAPI`/`Dio` clients for backend communication. The app integrates services such as `Firebase`, `Sentry`, analytics SDKs, ad networks, in-app purchases, image caching, remote master data, and dynamic localization to support a production mobile experience.

Overall, the project implements the full client-side experience for SNPIT, including user authentication, camera and media workflows, social and competitive photo features, rewards, inventory, marketplace-style interactions, localization, remote configuration, and platform-specific mobile services. Its architecture reflects a mature Flutter application with many interconnected user journeys and external service dependencies.

We use the [OWASP] (https://owasp.org/www-community/OWASP_Risk_Rating_Methodology) severity matrix to determine severity. See the documentation for more details.

Disclaimer

The Safe Edges team diligently endeavors to uncover as many vulnerabilities in the project within the specified time frame, however, it assumes no liability for the discoveries outlined in this document. Please note that a security audit conducted by the team does not constitute an endorsement of the underlying business or product. The audit was conducted within a predetermined timeframe, focusing solely on assessing the security aspects of the project.

Audit Details

Scope

The security audit covered the SNPIT mobile application, specifically the `snpit-app-develop/` codebase

Executive Summary

Issues found

Severity	Number of issues found
HIGH	1
LOW	1
INFORMATIONAL	3
Note	1
Total	6

Findings

High

[H-01] Location Spoofing Enables Reward Abuse By Intentionally Allowing iOS

Description: The app performs fake-location detection only on `Android` before camera initialization, with no equivalent check on `iOS`. The app also does not perform root/device-integrity detection, so rooted or tampered devices are not blocked before submitting capture metadata. The photo upload flow sends client-provided latitude and longitude values as optional request parameters. Backend verification confirms that these coordinates are accepted without server-side location validation.

When a user uploads a photo, the backend stores the submitted latitude and longitude and later geocodes them into `photo_address`. This derived location is used in reward logic, including `SnapOnLocation` `STP` bonuses and battle reward multipliers.

Because the backend treats client-provided GPS coordinates as authoritative, an attacker using a spoofed location, rooted/tampered device, or modified client can submit false coordinates. This allows location-based rewards and battle multipliers to be claimed without being physically present at the claimed location.

Affected Code:

`lib/app/camera_snap/components/snap/initialize/camera_initialize_mixin.dart`

```
if (Platform.isAndroid) {
 final isFakeLocation = await DetectFakeLocation().detectFakeLocation();
 if (isFakeLocation) {
 setState(() {
 errorMessage = l10n.homeLocationDisguiseError;
 errorDescription = l10n.homeLocationDisguiseErrorDesc;
 isLoading = false;
});
 return;
}
}
// iOS skips fake-location check
```

Impact: Attackers can use spoofed locations, rooted/tampered devices, or modified clients to submit false GPS coordinates and claim location-based rewards or battle multipliers without being physically present, enabling reward fraud and unfair gameplay advantages.

Recommended mitigation: Enforce server-side validation for submitted GPS coordinates using impossible-travel checks, rate limits, location-history consistency, and anomaly scoring. Add iOS fake-location detection and platform-specific device-integrity checks for both Android and iOS. Treat failed integrity checks as high-risk for reward-bearing actions.

Low

[L-01] Sentry Crash Reporting Includes User PII Without Proper Scrubbing

Description: The app configures Sentry crash reporting with minimal event filtering and sets the Sentry user scope to include personally identifiable information: user UUID, email address, and display name. When the app crashes or encounters an error, this user-identifying data is sent to Sentry's servers along with crash telemetry. The `beforeSend` callback exists for filtering but but the callback only drops `PathNotFoundException` events and does not scrub PII.

This exposes user email addresses and display names to Sentry storage and to project members with access to crash reports, increasing privacy and data-minimization risk.

Affected Code:

`lib/utils/sentry.dart`

```
await SentryFlutter.init((options) {
 options
..dsn = dsn
..environment = Config.buildTarget
..beforeSend = _beforeSend
..debug = false;
```

```
}, appRunner: () => runApp());
```

`lib/app/user/providers/user_provider.dart`

```
Sentry.configureScope((scope) {
 scope.setUser(
 SentryUser(
 id: state.uuid,
 email: state.email,
 username: state.displayName,
),
);
});
```

Impact: User email addresses may be sent to and stored in Sentry as part of crash reports, increasing the risk of unnecessary PII exposure.

Recommended mitigation: Remove email addresses from Sentry user scope. Use only internal Uuid and display name. Add PII scrubbing in `beforeSend` callback to remove emails from stack traces and request data.

Info

[I-01] Global iOS App Transport Security Bypass Enabled

Description: The iOS app sets `NSAllowsArbitraryLoads` to `true` in `Info.plist`, globally disabling App Transport Security restrictions. ATS is iOS's built-in transport security control that requires secure HTTPS connections and enforces minimum TLS standards for network requests.

With this global exception enabled, the app may load cleartext HTTP URLs or connect to endpoints with weaker transport security than ATS would normally allow. The endpoints in the codebase use HTTPS, so no current exploit path is confirmed.

This configuration may also create App Store review risk, because Apple expects ATS exceptions to be justified and narrowly scoped where possible.

Affected Code:

`ios/Runner/Info.plist`

```
<key>NSAppTransportSecurity</key>
<dict>
<key>NSAllowsArbitraryLoads</key>
<true/>
</dict>
```

Impact: Future HTTP endpoints or misconfigured URLs could potentially load without ATS protection, increasing the risk of cleartext traffic exposure or transport-layer interception.

Recommended mitigation: Remove global `NSAllowsArbitraryLoads`. If an exception is unavoidable, use narrowly scoped `NSExceptionDomains` for specific trusted domains and keep all API and WebView URLs on HTTPS.

[I-02] Marketplace One-Time JWT Passed Through URL Query Parameter

Description: The app requests a one-time JWT from the marketplace API and appends it to the marketplace authentication URL as a `token` query parameter. The URL is then opened in an in-app WebView. Placing bearer-like credentials in URLs is a security anti-pattern because URLs are commonly captured by URL-handling surfaces such as WebView history, server access logs, analytics, crash reports, referrers, or screenshots.

Affected Code:

`lib/app_core/constants/asset_urls.dart`

```
static String marketplaceAuthUrl(String jwt, String languageCode) {
 return '$marketplaceBaseUrl/auth?token=$jwt&lang=$languageCode';
}
```

`lib/app/market/utils/marketplace_launcher.dart`

```
final jwt = await ref
.read(marketplaceRepositoryProvider)
.getOneTimeJWT(walletAddress: selectedWalletAddress);
final marketplaceUrl = AssetUrls.marketplaceAuthUrl(jwt, languageCode);
final uri = Uri.parse(marketplaceUrl);
await launchUrl(uri, mode: LaunchMode.inAppWebView);
```

Impact: The one-time JWT may be exposed through URL-handling surfaces. If captured before redemption or expiry, it can potentially be used to authenticate a marketplace web session as the victim.

Recommended mitigation: Avoid passing tokens in URL parameters; use a backend session exchange or POST-based redemption flow instead. Make marketplace JWTs short-lived, single-use, and invalidate them immediately after use.

[I-03] Plaintext Local Storage of Access Tokens and Location Metadata

Description: The app stores sensitive local data without using platform secure storage. The API bearer token is persisted through `SharedPreferences`, and offline capture data is stored under the app Documents directory with JSON metadata containing token ID, timestamp, angle, and precise GPS coordinates.

If local app data is extracted through device compromise, malware, physical access, or backup/forensic tooling, an attacker may recover the bearer token or offline location metadata. This is a local data protection hardening issue rather than a remotely exploitable vulnerability.

Affected Code:

`lib/infrastructure/local_user_access_token/local_user_access_token.dart`

```
class LocalUserAccessToken extends BaseSharedPreferencesService<String> {
@override
 SharedPreferencesKey get key => SharedPreferencesKey.accessToken;
@override
 Future<void> set(String value) {
 ref.read(apiClientProvider).setAccessToken(value);
 return super.set(value);
}
}
```

`lib/features/offline/offline_model.dart`

```
meta.add({
'id': id,
'fileName': fileName,
'tokenId': tokenId,
'angle': angle,
'latitude': latitude,
'longitude': longitude,
'takenAt': DateTime.now().toIso8601String(),
});
```

```
final base = await getApplicationDocumentsDirectory();
final dir = Directory(p.join(base.path, _dirName));
```

Impact: If local app storage is extracted, access tokens and precise location metadata may be exposed, enabling API authentication as the victim until token expiry/revocation or revealing the users capture locations.

Recommended mitigation: Store bearer tokens in iOS Keychain or Android Keystore-backed secure storage. Encrypt sensitive offline metadata where feasible and clear sensitive offline state on logout and account deletion.

Note

[N-01] Privy Wallet Linking Could Potentially Accept Unverified WebView JavaScript Messages

Description: The Privy wallet-linking flow opens a remote URL in `CommonWebView`, registers a JavaScript channel named `FlutterPrivyLoginEventsListener`, and treats the received message as a wallet address without client-side origin, schema, or format validation. The message is passed to `integratePrivy(address: address)`, which submits the address to the backend for wallet integration.

The vulnerable component is the client-side trust boundary: remote WebView content controls the wallet address submitted to the backend. If an attacker can influence the loaded WebView content or navigation target, they could cause the app to submit an attacker-chosen wallet address. However, the current codebase does not prove that an external attacker can control the production Privy page.

Affected Code:

`lib/app_core/ui_parts/common_web_view.dart`

```
await _controller.setJavaScriptMode(JavaScriptMode.unrestricted);
await _controller.addJavaScriptChannel(
 entry.key,
 onMessageReceived: (message) {
 entry.value(message.message);
},
);
```

`lib/app/wallet/privy_connect_page.dart`

```
CommonWebView(
 url: url,
 javaScriptChannelActions: {
_javaScriptChannelName: (message) {
 log('onJavaScriptMessage: $message');
```

```
 if (context.mounted) {
 context.pop(message);
}
},
},
)
```

`lib/app/wallet/components/snpit_wallet_card.dart`

```
final address = await PrivyConnectPage.show(context);
if (address == null) {
 return;
}
await ref
.read(externalWalletRepositoryProvider)
.integratePrivy(address: address);
```

`lib/repository/external_wallet/external_wallet_repository.dart`

```
Future<void> integratePrivy({required String address}) async {
 final response = await _externalWalletApi
.externalWalletControllerIntegratePrivyWallet(address: address);
}
```

Impact: An unverified wallet address can be linked to the authenticated user account. If the WebView message or client request is influenced, the backend will accept the submitted Privy address, allowing incorrect or unauthorized wallet binding.

Recommended mitigation: Require backend-side wallet ownership verification before linking, such as a signed nonce challenge. Apply strict address format validation and duplicate checks as secondary controls.